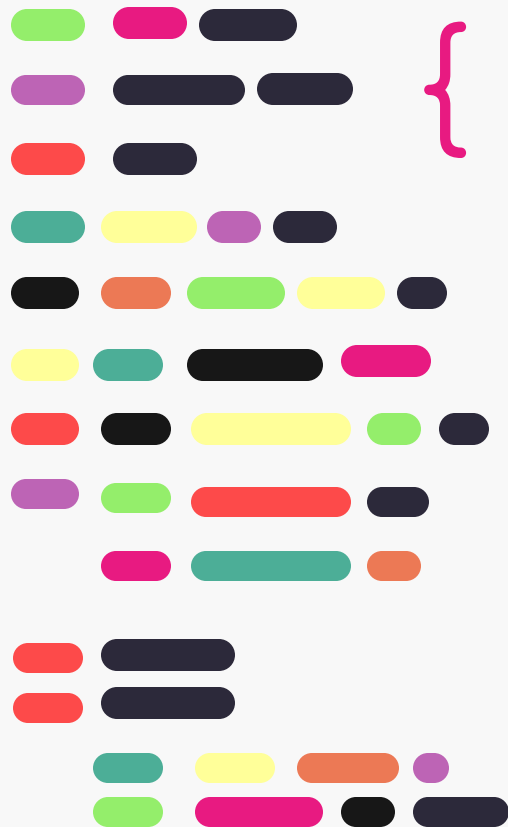




Conception Orientée Objet & Programmation JAVA

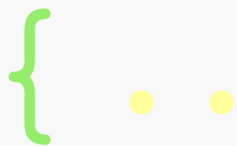
Chapitre 6: Polymorphisme



Objectifs du chapitre



- Comprendre les notions de surcharge, de substitution et leur rôle dans le polymorphisme.
- Découvrir le transtypage (ou conversion de type, appelée cast en anglais).



Polymorphisme



Polymorphisme

Le polymorphisme désigne la capacité d'un même élément – qu'il s'agisse d'une méthode ou d'un attribut – à se comporter de différentes manières selon le contexte d'utilisation ou le type de l'objet concerné.

Le terme polymorphisme vient du grec ancien et signifie « qui peut prendre plusieurs formes », ce qui illustre parfaitement son rôle en programmation orientée objet.

Polymorphisme

En Java, le polymorphisme est rendu possible principalement grâce à deux mécanismes fondamentaux :

1.L'héritage, qui permet à une sous-classe de redéfinir ou d'adapter le comportement hérité d'une superclasse.

2.Les interfaces, qui offrent un moyen de définir des comportements communs pouvant être implémentés différemment par plusieurs classes.

Polymorphisme & Héritage

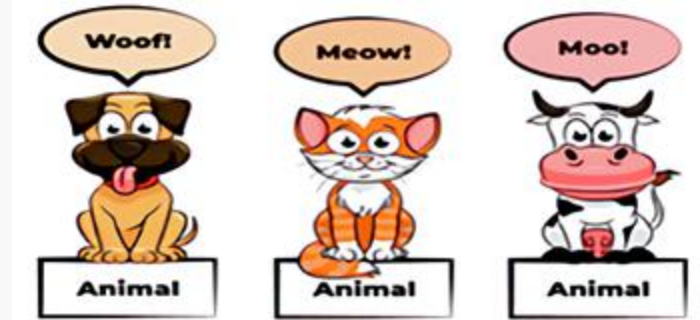
- L'**héritage** permet à une sous-classe d'hériter des **attributs** et **méthodes** d'une classe parente.
- La sous-classe peut **redéfinir** ces méthodes pour **adapter leur comportement** (*overriding*).
- Le **polymorphisme par héritage** permet à un **objet de sous-classe** d'être utilisé comme un **objet de la classe parente**.

→ Offre une **grande flexibilité** et favorise un **code réutilisable et générique**.

Polymorphisme (Exemple)

```
class Animal {  
  
    public void talk() {  
        System.out.println("Je suis un  
animal !");  
    }  
}
```

```
class Cat extends Animal {  
  
    @Override  
    public void talk() {  
        System.out.println("Meow !");  
    }  
  
    public void scratch() {  
        System.out.println("je griffe !");  
    }  
}
```



```
class Dog extends Animal {  
  
    @Override  
    public void talk() {  
        System.out.println("Ouaf !");  
    }  
  
    public void attack() {  
        System.out.println("j'attaque !");  
    }  
}
```

Polymorphisme (Exemple)

```
public class AnimalTester {  
  
    public static void main(String[] args) {  
        Dog monChien = new Dog();  
        monChien.talk();  
        monChien.attack();  
  
        Cat monChat = new Cat();  
        monChat.talk();  
        monChat.scratch();  
  
        Animal monAnimal = new Dog(); //Instance de Dog déclarée avec référence Animal  
        monAnimal.talk();  
  
        Animal monAnimal2 = new Cat(); //Instance de Cat déclarée avec référence Animal  
        monAnimal2.talk();  
  
        monAnimal.attack(); //cause une erreur de compilation  
        monAnimal2.scratch(); //cause une erreur de compilation  
    }  
}
```


Polymorphisme (Exemple)

Le compilateur se base sur le type de référence :

→ **monChien** → méthode **talk()** de **Dog**

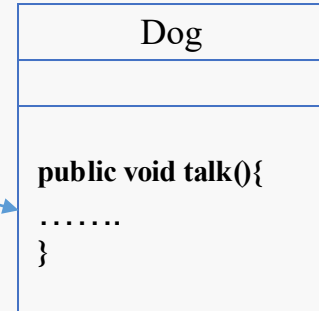
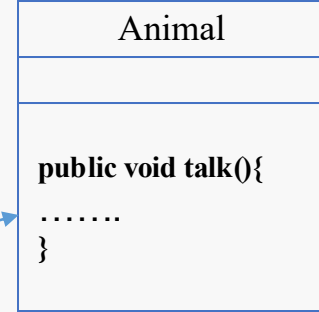
→ **monAnimal** → méthode **talk()** de **Animal**

La JVM, lors de l'exécution, appelle la méthode redéfinie dans la classe réelle de l'objet → ici Chien.

NB : Le choix de la méthode n'est pas fait à la compilation (liaison statique) mais au moment de l'exécution (liaison dynamique).

Polymorphisme (Exemple)

```
public class AnimalTester {  
    public static void main(String[] args) {  
        Animal monAnimal = new Dog();  
        monAnimal.talk();  
    }  
}
```



**==> Au moment de l'exécution, c'est le niveau
le plus bas qui gagne.**

Polymorphisme (Exemple)

```
public class AnimalTester {  
    public static void main(String[] args) {  
        Animal monAnimal = new Dog();  
        monAnimal.attack();  
    }  
}
```

L'appel de la méthode **monAnimal.attack()** cause une erreur car le compilateur considère que **monAnimal** est un **Animal** et **non pas un Dog**, et donc ne possède pas de méthode nommée `attack()`, bien qu'il contient une instance de Chien.

Polymorphisme (Exemple)

l'objet `monAnimal` est dit **surclassé** puisqu'il est déclaré avec la référence de la classe `Animal`, qui ne possède pas la méthode `attack()`.

Il faut donc **substituer** `monAnimal` par :

`((Dog) monAnimal).attack();`

La solution est de “convertir” le type de

l'objet `monAnimal` à `Dog` pour pouvoir utiliser la méthode `attack()`.

```
public class AnimalTester {  
    public static void main(String[]  
args) {  
        Animal monAnimal = new Dog();  
        ((Dog) monAnimal).attack();  
    }  
}
```

Polymorphisme

Cacher une méthode static

- En **Java**, une **méthode statique** ne peut pas être **redéfinie** dans une sous-classe.
 - Cependant, elle peut être « **cachée** » si la sous-classe déclare une **méthode statique** portant la **même signature** que celle de la superclasse.
 - Ce phénomène est appelé **hiding** (*masquage*), et non **overriding** (*redéfinition*).
- ⚙ Le **choix de la méthode statique** à exécuter est fait à la **compilation**, contrairement aux **méthodes d'instance**, résolues à l'**exécution**.

Cacher une méthode static : exemple 1

```
public class Animal {  
  
    public static void run() {  
        System.out.println("An animal runs");  
    }  
}  
  
public class Cat extends Animal{  
  
    public void run() { //Erreur de compilation (interdit de redéfinir une méthode statique)  
        System.out.println("A cat runs");  
    }  
}
```

Cacher une méthode static : exemple 2

```
public class Animal {  
  
    public static void run() {  
        System.out.println("An animal runs");  
    }  
}  
  
public class Cat extends Animal{  
  
    public static void run() {  
        super.run(); //Erreur de compilation (interdit d'appeler une méthode statique avec  
super)  
        System.out.println("A cat runs");  
    }  
}
```

Cacher une méthode static : exemple 3

On peut appeler la méthode caché en convertissant l'objet **monChat** en un **Cat** ou en appelant la méthode directement par le nom de la classe.

```
public class AnimalTester {  
  
    public static void main(String[] args){  
        Animal monAnimal = new Animal();  
        Animal monChat = new Cat();  
  
        monAnimal.run(); //An animal runs  
        monChat.run(); //An animal runs  
  
        ((Cat) monChat).run(); //A cat runs  
        Cat.run(); //A cat runs  
  
    }  
}
```


Polymorphisme

Tableaux polymorphiques

- Un **tableau polymorphique** peut contenir des **objets de différentes classes** qui **héritent d'une même classe parente**.
- Il permet de **regrouper plusieurs types d'objets** partageant un **comportement commun**.

Avantage : écrire du **code générique et flexible**, capable de traiter des objets variés via leurs **méthodes héritées** ou **redéfinies**.

```
Animal[] animaux = new
Animal[2];
animaux[0] = new Dog();
animaux[1] = new Cat();

for (Animal animal : animaux)
{
    animal.talk(); // appelle
    la méthode talk() de chaque type
    d'objet, soit talk() de la
    classe Dog ou talk() de la classe
    Cat.
}
```

Upcast & Downcast

En Java, il existe **deux types de cast** (*conversion de type*) : **Upcast** et **Downcast**, pouvant être **implicites** ou **explicites**.

1. Upcast

- Conversion d'un **objet de sous-classe** vers sa **super-classe**.
- Effectué **implicitement** (automatique).

```
Animal a = new Chien(); // Upcast implicite
```

2. Downcast

- Conversion d'un **objet de super-classe** vers une **sous-classe**.
- Nécessite un **cast explicite**.

```
Chien c = (Chien) a; // Downcast explicite
```

Attention : Un downcast incorrect peut provoquer une **ClassCastException** à l'exécution.

Upcast & Downcast

```
public class AnimalTester {  
  
    public static void main(String[] args) {  
  
        Animal monAnimal = new Dog(); //Upcast implicite  
        Animal monAnimal2 = (Animal) new Dog(); //Upcast explicite  
  
        Cat monChat = new Animal(); //Downcast implicite  
        Dog monChien = (Dog) new Animal(); //Downcast explicite  
  
    }  
}
```

Upcast

- La conversion vers une super-classe est appelée upcast.
- Elle peut être implicite ou explicite, mais dans les deux cas :
 - > Elle est toujours acceptée par le compilateur.
 - > Elle ne provoque aucun problème à l'exécution.

L'**upcast** permet de manipuler un objet de sous-classe à travers les attributs et méthodes définis dans la classe parente.

```
public class AnimalTester {  
  
    public static void main(String[] args) {  
  
        Animal monAnimal = new Dog();  
        Animal monAnimal2 = (Animal) new Dog(); (Animal) est optionnel  
    }  
}
```

Downcast

- Le **downcast implicite** est **interdit en Java**.
- Il doit être effectué **explicitement** à l'aide d'un **cast** pour éviter les erreurs de compilation.

Même s'il est accepté par le compilateur, un **downcast incorrect** peut provoquer une **erreur à l'exécution** (*ClassCastException*).

```
public class AnimalTester {  
  
    public static void main(String[] args) {  
        Animal monAnimal = new Dog();  
        Dog monChien = monAnimal; //génère une erreur de compilation  
    }  
}
```

Downcast

- Le **downcast explicite** est toujours accepté par le compilateur.
- **Attention à l'exécution** : il peut échouer si l'objet n'est pas une instance de la sous-classe cible.
- Une **ClassCastException** sera alors levée.

```
public class AnimalTester {  
  
    public static void main(String[] args) {  
        Animal animal = new Dog();  
        Dog chien = (Dog) animal;  
        Wolf loup = (Wolf) animal; //génère une exception à l'exécution  
    }  
}
```

Downcast

- Après un **downcast**, l'objet est **vu comme une instance de la sous-classe**.

Exemple :

```
public class AnimalTester {  
    public static void main(String[] args) {  
        Animal monAnimal = new Dog();  
        monAnimal.talk(); // Output: "Ouaf !"  
        Dog monChien = (Dog) new Animal(); // Erreur d'exécution  
        monChien.talk();  
        monChien.attack();  
    }  
}
```

Conséquence : on peut accéder aux **méthodes propres à la sous-classe** qui n'étaient pas visibles via la référence de la super-classe.

Downcast

- Un **downcast** peut **échouer à l'exécution** si l'objet de la super-classe **n'est pas une instance de la sous-classe cible**.
 - Dans ce cas, une **ClassCastException** est levée.
 - Le **compilateur** n'émet **aucune erreur**, car le cast explicite est syntaxiquement correct.

```
public class AnimalTester {  
    public static void main(String[] args) {  
        Animal animal = (Dog) new Animal(); //Erreur d'exécution (ClassCastException)  
    }  
}
```


Transtypage des types primitifs

La conversion entre types primitifs permet de convertir la valeur d'un type en un autre type primitif. Cela se produit le plus souvent avec les types numériques.

```
public class Main{  
  
    public static void main(String[] args) {  
  
        float floatValue = 123.45f;  
        int intValue = (int) floatValue;  
        System.out.println(intValue); //Output: 123 ⇒ Perte de données  
        String stringValue = (String) intValue; //Erreur de compilation (types incompatibles)  
        Integer integerValue = null;  
        intValue = (Integer) integerValue;  
        System.out.println(integerValue); //Erreur à l'exécution (NullPointerException)  
  
    }  
}
```

Lorsqu'on fait l'affectation d'un entier long (64 bits) à un entier int (32 bits), ou d'un réel double vers un réel float, on risque de perdre une partie de données.

L'opérateur instanceof

- `instanceof` permet de **vérifier si un objet est une instance** :
 - d'une **classe spécifique**
 - ou d'une **sous-classe** de cette classe.
- Utile pour **adapter le comportement** selon le type réel de l'objet avant d'effectuer un **downcast** ou une opération spécifique.

```
public class AnimalTester {  
  
    public static void main(String[] args) {  
        for (Animal animal : animaux) {  
            if (animal instanceof Dog)  
                ((Dog) animal).attack();  
            if (animal instanceof Cat)  
                ((Cat) animal).scratch();  
        }  
    }  
}
```

Nb: l'opérateur instanceof ne fonctionne qu'avec des objets et des classes, pas avec des types primitifs tels que int ou double.

L'opérateur instanceof

- Depuis **Java 16**, vous pouvez **créer une variable locale directement après instanceof**.
- Le **downcast** est automatique, ce qui rend le code **plus simple et sûr**.

```
public class AnimalTester {  
  
    public static void main(String[] args) {  
  
        for (Animal animal : animaux) {  
            if (animal instanceof Dog dog)  
                dog.attack();  
            if (animal instanceof Cat cat)  
                cat.scratch();  
        }  
    }  
}
```

La méthode getClass()

- `getClass()` retourne la **classe réelle** d'un objet au moment de l'exécution.
- Utile pour connaître le **type exact** d'un objet, indépendamment du type de sa référence.

```
public class AnimalTester {  
  
    public static void main(String[] args) {  
        Animal animal = new Animal();  
        Animal chien = new Dog();  
        System.out.println(animal.getClass() == Animal.class); //true  
        System.out.println(chien.getClass() == Animal.class); //false  
        System.out.println(chien instanceof Dog); //true  
        System.out.println(chien instanceof Animal); //true  
    }  
}
```

La méthode equals()

La méthode `equals()` en Java

- Héritée de la **classe Object**, `equals()` permet de **comparer deux objets**.
- **Par défaut** : retourne `true` uniquement si les deux références pointent vers **le même objet en mémoire**.
- Pour comparer les **contenus** des objets, il faut **redéfinir** (override) la méthode dans la classe concernée.

```
public class AnimalTester {  
    public static void main(String[] args) {  
        Animal animal1 = new Animal("Snoopy", 2);  
        Animal animal2 = new Animal("Snoopy", 2);  
        //Avant la redéfinition  
        System.out.println(animal1.equals(animal2)); //false  
        //Après la redéfinition  
        System.out.println(animal1.equals(animal2)); //true }  
}
```

On suppose qu'un animal est caractérisé par un nom et un âge.

La méthode equals(): exemple 1

Il existe plusieurs implémentations possible pour la méthode equals.

```
public class Animal {  
  
    @Override  
    public boolean equals(Object obj) {  
        if (null == obj) return false;  
        if (obj instanceof Animal animal){  
            return age == animal.age && name.equals(animal.name);  
        }  
  
        return false;  
    }  
}
```

Nb: La méthode equals est redéfinie dans la classe String. Elle permet de comparer 2 chaînes de caractères.

La méthode equals(): exemple 2

```
public class Animal {  
    @Override  
    public boolean equals(Object obj) {  
        if (this == obj) return true;  
        if (null == obj) return false;  
        if (obj.getClass() == Animal.class){  
            Animal animal=(Animal) obj;  
            return age == animal.age && name.equals(animal.name);  
        }  
        return false;  
    }  
}
```

{ • • Merci pour votre attention

